④ Gets the session for user gamer1

⑤ Waits 15 seconds and gets the session for gamer1 again

⑥ The response is empty, because the key was automatically deleted after 15 seconds.

⑦ Adds gamer1 with a score of 100 to the sorted set named highscore

⑧ Adds gamer2 with a score of 50 to the sorted set named highscore

⑨ Adds gamer3 with a score of 150 to the sorted set named highscore

⑩ Adds gamer4 with a score of 5 to the sorted set named highscore

Gets the top three gamers from the highscore list

The list is sorted ascending and includes the user followed by its score.

Quits the Redis CLI

MemoryDB behaves like ElastiCache for Redis, but it comes with persistence guarantees, which make it possible to use it as the primary database.



Cleaning up

It's time to delete the CloudFormation stack named `memorydb-minimal`. Use the AWS Management Console or the following command to do so:

```
$ aws cloudformation delete-stack --stack-name memorydb-minimal.
```

You are now equipped to select the best-fitting in-memory database engine and deployment option for your use case. In the next section, you will take a closer look at the security aspects of ElastiCache to control access to your cache cluster.

11.3 Controlling cache access

Controlling access to data stored in ElastiCache is very similar to the way it works with RDS (see section 11.4). ElastiCache is protected by the following four layers:

- *Identity and Access Management (IAM)*—Controls which IAM user, group, or role is allowed to administer an ElastiCache cluster.
- *Security groups*—Restricts incoming and outgoing traffic to ElastiCache nodes.
- *Cache engine*—Redis >6.0 supports authentication and authorization with role-based access control (RBAC). Memcached does not support authentication.
- *Encryption*—Optionally, data can be encrypted at rest and in transit.